

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09

NOMBRE COMPLETO: González Armas Dylan Arturo

Nº de Cuenta: 419049513

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 28/10/2026

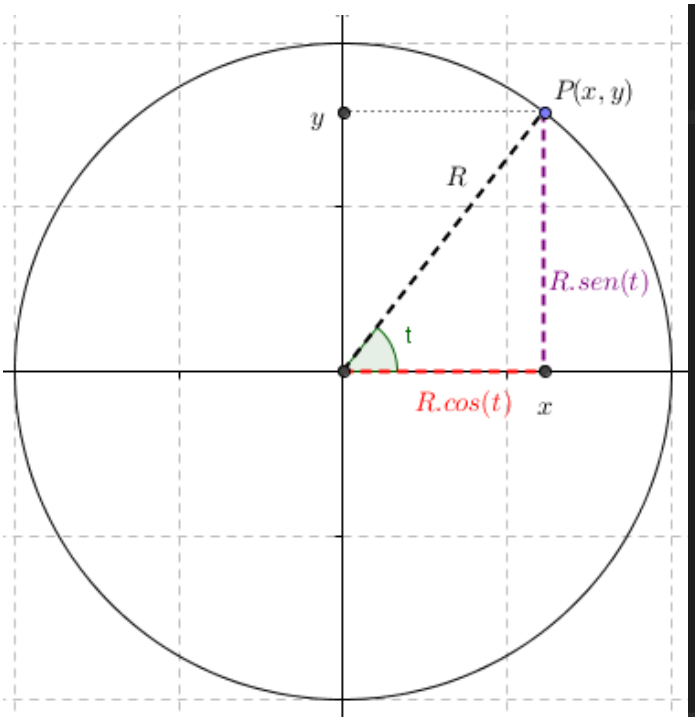
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio del satélite.

Para el ejercicio del satélite modifique lo obtenido con el circuito del avión y solo se creo una estructura de control más sencilla utilizando únicamente la ecuación paramétrica d ela circunferencia como algoritmo.




```
switch (estadoSatelite)
{

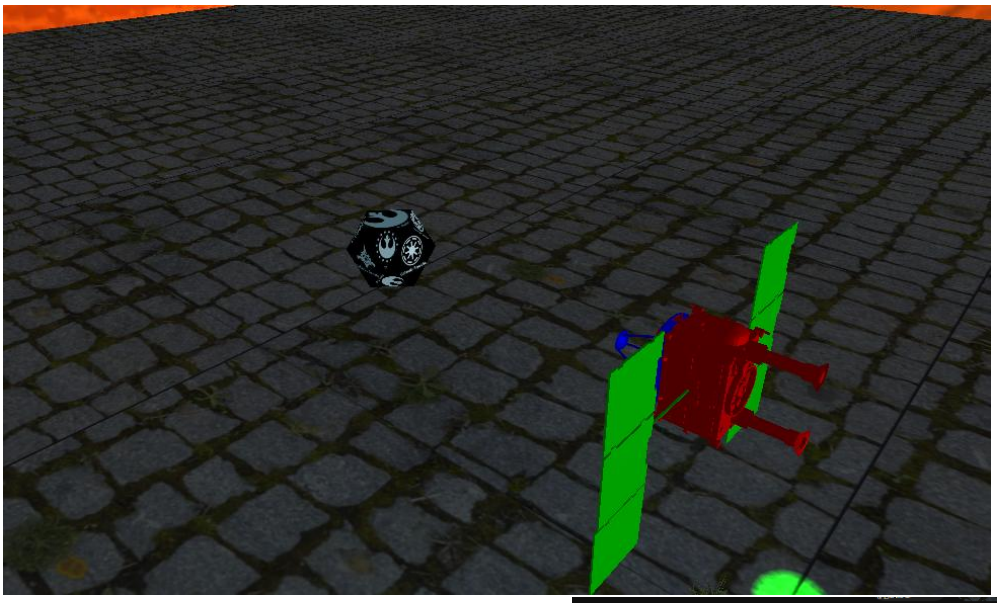

case 0: // 2. Primer giro de 1/4 de circunferencia (0° a 90° -> gira hacia +X)
    if (anguloSat < 90.0f)
    {
        anguloSat += velSatGiro * deltaTime;
        // Centro del arco 1: Cx = radioCurva, Cz = -tramoLargoZ
        movSatX = radioCurva2 - radioCurva2 * cos(anguloSat * toRadians);
        movSatZ = radioCurva2 - radioCurva2 * sin(anguloSat * toRadians);
        rotSatY = -anguloSat; // Gira la nariz del avión junto con el arco
    }
    else
    {
        anguloSat = 90.0f;
        movSatX = radioCurva2;
        movSatZ = radioCurva2 - radioCurva2;
        rotSatY = -90.0f;
        estadoSatelite = 1;
    }
    break;

case 1:
    if (anguloSat < 180.0f)
    {
        anguloSat += velSatGiro * deltaTime;
        // Centro del arco 2: Cx = radioCurva + tramoCortoX, Cz = -tramoLargoZ
        movSatX = (radioCurva2) - radioCurva2 * cos(anguloSat * toRadians);
        movSatZ = radioCurva2 - radioCurva2 * sin(anguloSat * toRadians);
        rotSatY = -anguloSat;
    }
    else
    {
        anguloSat = 180.0f;
        movSatX = (2.0f * radioCurva2);
        movSatZ = 0;
        rotSatY = -180.0f;
        estadoSatelite = 2;
    }
    break;

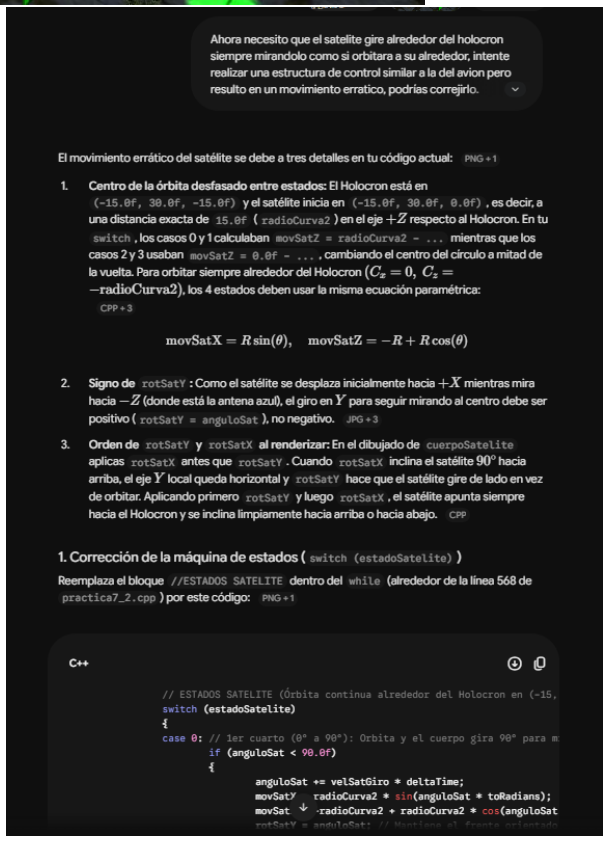

case 2:
    if (anguloSat < 270.0f)
    {
        anguloSat += velSatGiro * deltaTime;
        movSatX = (radioCurva - radioCurva2 * cos(anguloSat * toRadians));
        movSatZ = 0.0f - radioCurva2 * sin(anguloSat * toRadians);
        rotSatY = -anguloSat;
```

Desafortunadamente mi solución no tuvo buenos resultados dado que mi idea era realizar una rotación alrededor del holocron como si fuera una estrella utilizando el satélite como objeto para animar pero la animación resultante era errónea y no pude dar el efecto que buscaba por fases los cuales quería que fueran las siguientes:

1. **El satélite comienza mirando directamente al holocron y en el primer cuarto de circunferencia gira a su alrededor mientras rota 90 grados terminando mirando hacia arriba.**
2. **Luego en el segundo ciclo los paneles solares giran lentamente hasta quedar perpendiculares al holocron mirando la figura directamente de nuevo.**
3. **Luego en el penúltimo cuarto el cuerpo del satélite gira 180 grados para terminar mirando hacia abajo mientras la antena rota sobre su propio eje.**
4. **Por último en el cuarto final el satélite lentamente gira para volver a la posición en la que arranco y el ciclo vuelve a repetirse indefinidamente.**



Emplee a Gemini en modelo 3.6 flash para solucionar los errores de animación que presentaban mi estructura de control.



Ahora necesito que el satelite gire alrededor del holocron siempre mirandolo como si orbitara a su alrededor, intente realizar una estructura de control similar a la del avion pero resulto en un movimiento erratico, podrías correjirlo.

El movimiento errático del satélite se debe a tres detalles en tu código actual: PNG+1

1. **Centro de la órbita desfasado entre estados:** El Holocron está en `(-15.0f, 30.0f, -15.0f)` y el satélite inicia en `(-15.0f, 30.0f, 0.0f)`, es decir, a una distancia exacta de `15.0f` (`radioCurva2`) en el eje $+Z$ respecto al Holocron. En tu `switch`, los casos 0 y 1 calculaban `movSatZ = radioCurva2 - ...` mientras que los casos 2 y 3 usaban `movSatZ = 0.0f - ...`, cambiando el centro del círculo a mitad de la vuelta. Para orbitar siempre alrededor del Holocron ($C_x = 0$, $C_z = -\text{radioCurva2}$), los 4 estados deben usar la misma ecuación paramétrica: CPP+3

$$\text{movSatX} = R\sin(\theta), \quad \text{movSatZ} = -R + R\cos(\theta)$$

2. **Signo de `rotSatY`:** Como el satélite se desplaza inicialmente hacia $+X$ mientras mira hacia $-Z$ (donde está la antena azul), el giro en Y para seguir mirando al centro debe ser positivo (`rotSatY = anguloSat`), no negativo. JPG+3
3. **Orden de `rotSatY` y `rotSatX` al renderizar:** En el dibujado de `cuerpoSatelite` aplicas `rotSatX` antes que `rotSatY`. Cuando `rotSatX` inclina el satélite 90° hacia arriba, el eje Y local queda horizontal y `rotSatY` hace que el satélite gire de lado en vez de orbitar. Aplicando primero `rotSatY` y luego `rotSatX`, el satélite apunta siempre hacia el Holocron y se inclina limpiamente hacia arriba o hacia abajo. CPP

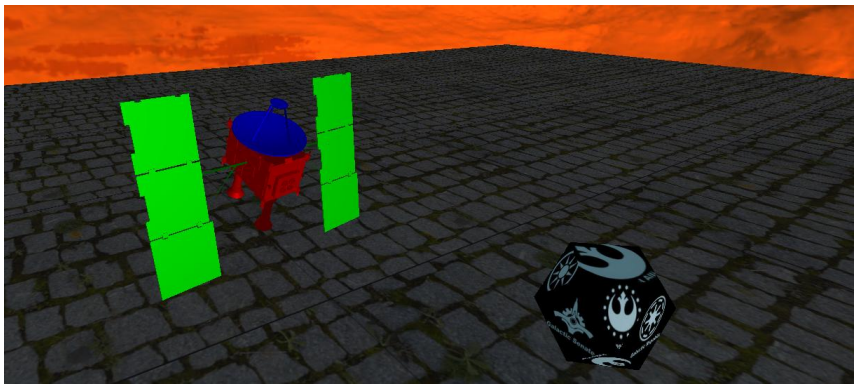
1. Corrección de la máquina de estados (`switch (estadoSatelite)`)

Reemplaza el bloque `//ESTADOS SATELITE` dentro del `while` (alrededor de la línea 568 de `practica7_2.cpp`) por este código: PNG+1

C++

```
// ESTADOS SATELITE (Órbita continua alrededor del Holocron en (-15,
switch (estadoSatelite)
{
case 0: // 1er cuarto (0° a 90°): Orbita y el cuerpo gira 90° para m
        if (anguloSat < 90.0f)
        {
                anguloSat += velSatGiro * deltaTime;
                movSatY   = radioCurva2 * sin(anguloSat * toRadians);
                movSat    = -radioCurva2 + radioCurva2 * cos(anguloSat
```

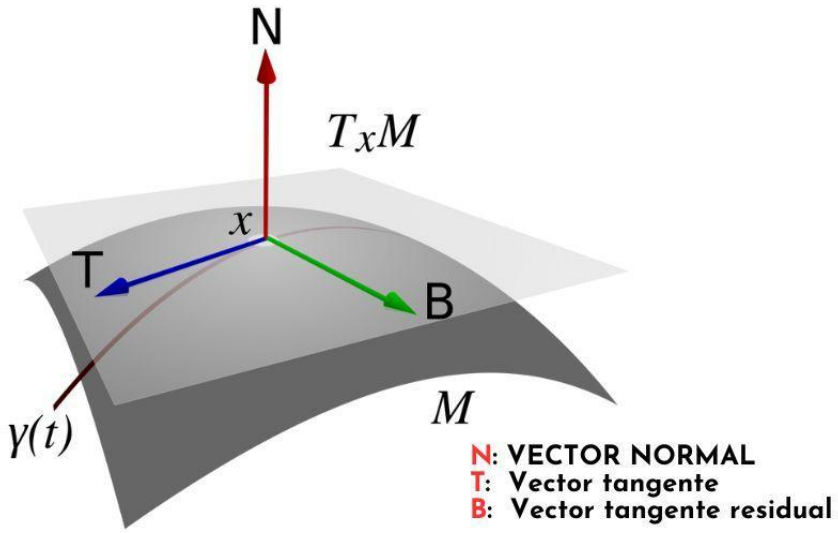
El resultado de este ejercicio se puede apreciar con mayor claridad en el video evidencia anexado en el reporte.

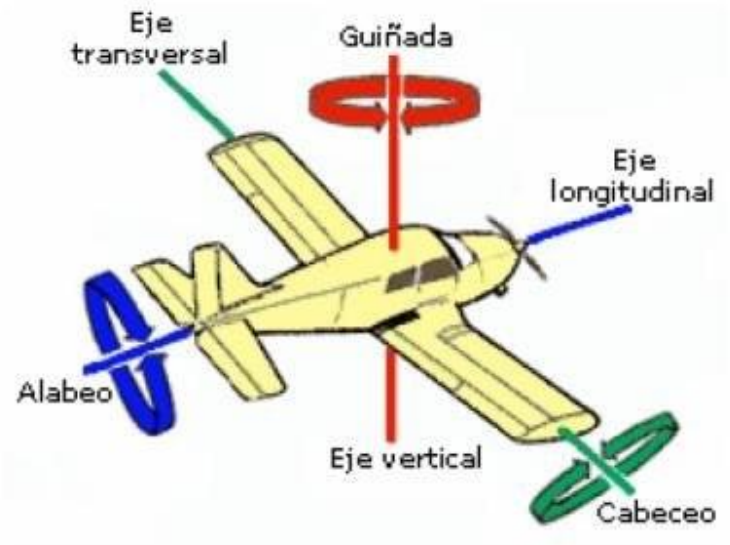


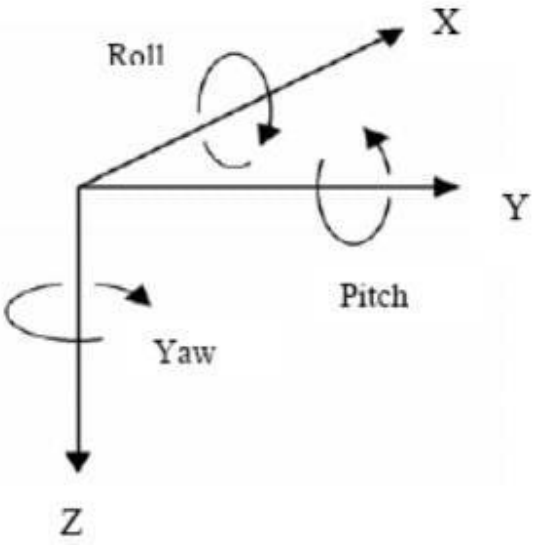
EJERCICIO DE PISTA RESUELTO POR MEDIO DE PATHING (ANIMACIÓN POR WAYPOINTS)

Para la resolución de este ejercicio opte por una metodología diferente, en este caso en vez de usar algoritmos algebraicos para obtener un trazado definido a partir de incrementos en traslaciones se utilizan dos vectores vec 3 para definir varios puntos que forman un circuito a lo largo de la pista.

Para este metodo se hace uso de un vector normal que siempre apunta de manera perpendicular a la superficie sobre la que este se encuentra, esto es importante porque permite que la nave ladeé de forma correcta y su giro al desplazarse sea realista ya que sin estos datos los coches solo seguirían la pista sin girar el modelo y la geometría de los dos chocaría dando un mal resultado



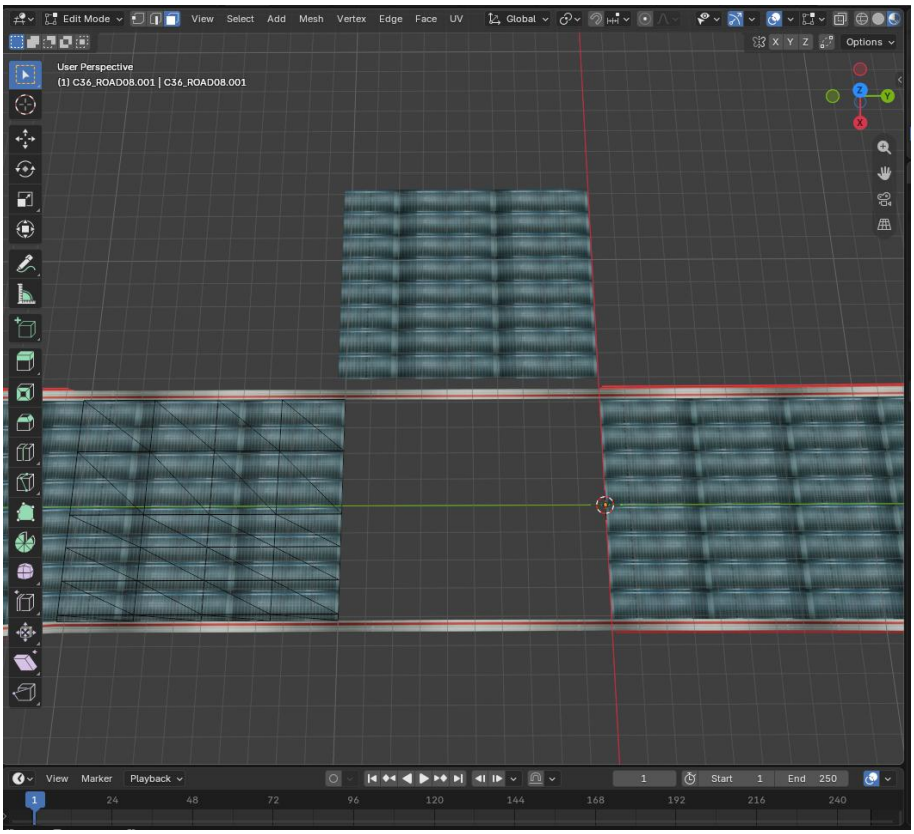




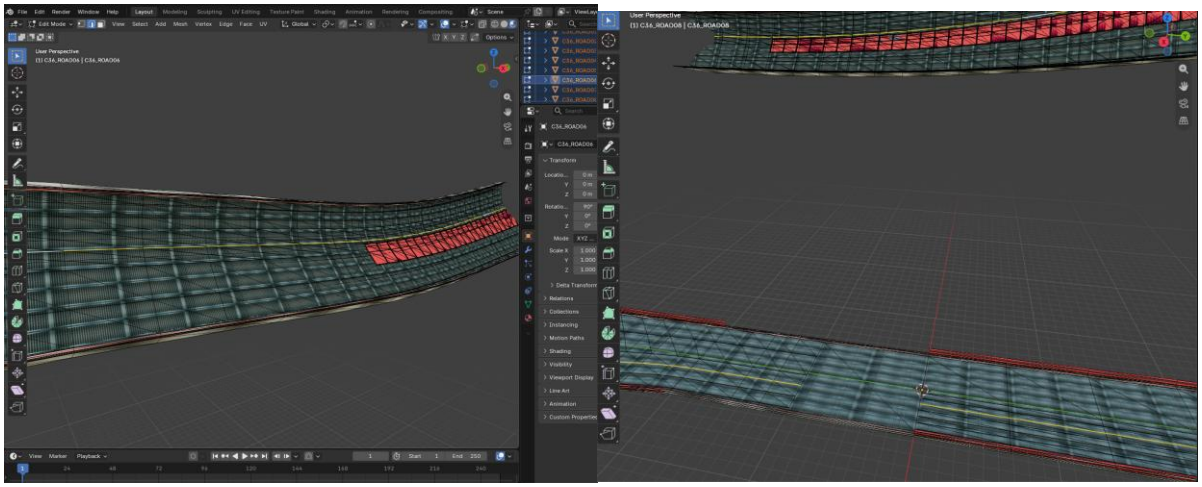
Por lo tanto para esta actividad explicaré primero mi trabajo que realicé yo y luego expondré la asistencia que tuve con inteligencia artificial.

Trabajo Manual

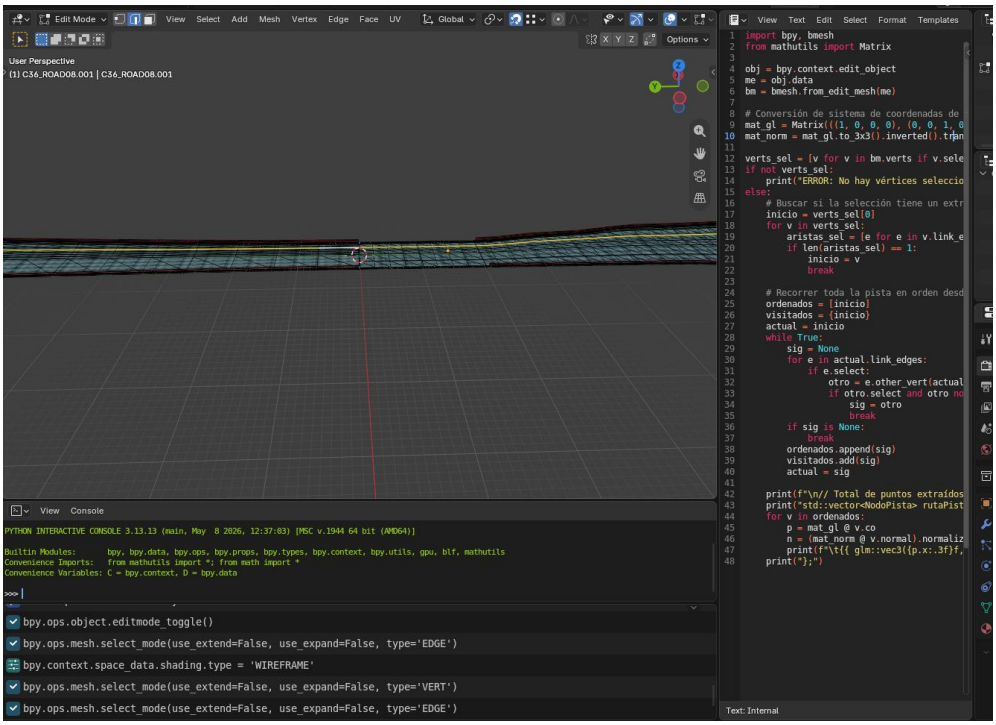
Una de las principales complicaciones que pueden surgir al animar por waypoints es que se hace uso de los vértices y si la malla del objeto 3D que se quiere navegar no es útil esto puede imposibilitar construir un camino continuo, por ello para hacer el trazado corregí partes de la pista cuya geometría estaba compuesta por triángulos demasiado grandes que no permitían hacer un camino suave ya sea cortando y remplazando parte de la geometría o creando nuevos vértices a lo largo de la malla para obtener un circuito de waypoints convincente que diera un buen efecto visual.



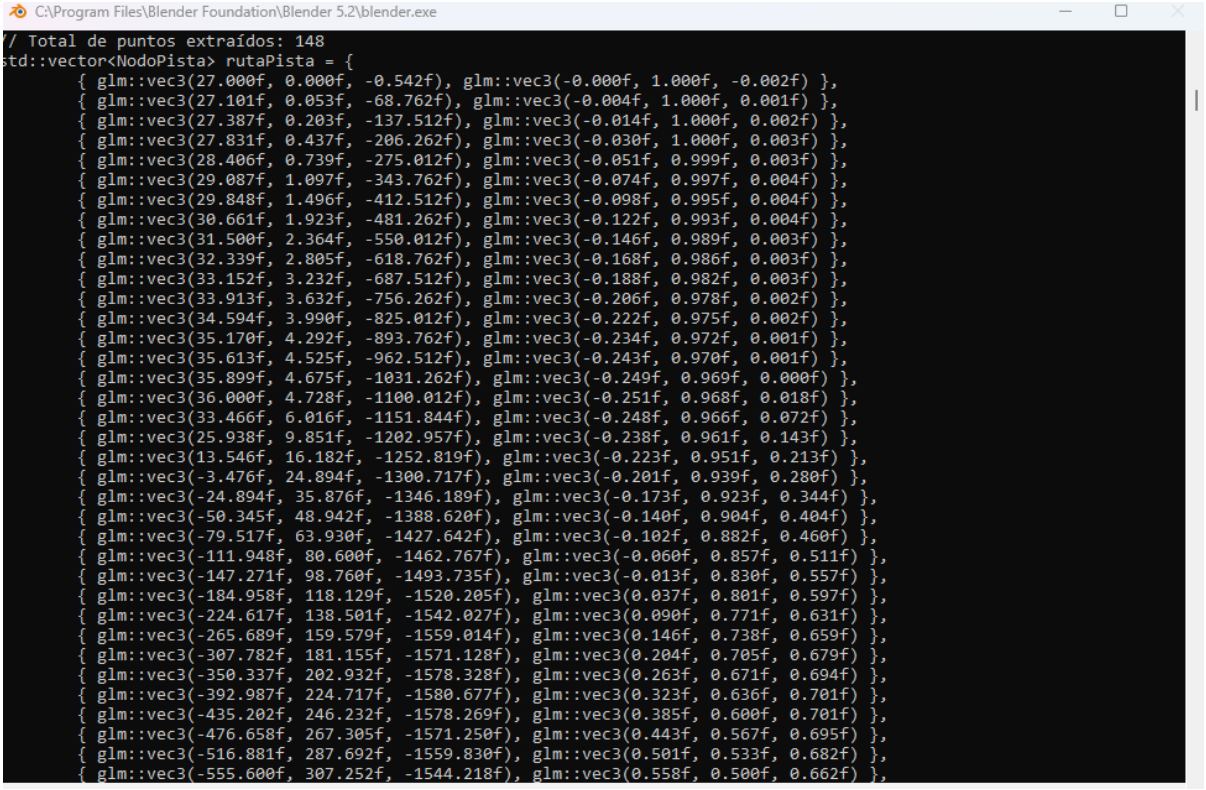
Una vez corregida la malla empecé a seleccionar los edge y vértices que conformarían el trazado final.



Teniendo cuidado de que no hubiera superposición de vértices ya que eso implicaría que el modelo se quedaría atascado durante una fracción de segundo en un punto antes de seguir el recorrido.



Una vez terminado el proceso de selección de vértices use un script de python solicitado por IA para extraer los vértices seleccionados y acomodarlos en un array de 2 vectores vec3 en una estructura llamada “NodoPista”



La parte con IA

Esta fue la sección donde más recibí asistencia de IA, ahora su trabajo fue ayudarme con la interpretación y procesamiento de los datos exportados en blender para hacer la animación de las naves fzero en pista. Básicamente se creó la estructura de control necesaria y se modificó el proceso de renderizado de los vehículos para aplicar los movimientos generados al implementar waypoints.

En tu `main.cpp`, observa que `modelaux` tiene una escala de `0.45f`, y luego a `pistaM` se le aplica otro `glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f))` antes de hacer `pistaM.RenderModel()`, mientras que las naves parten directamente de `modelaux`. El código a continuación ya multiplica los puntos de `pista.obj` por `0.45f` para que coincidan con el espacio de `modelaux`. CPP +1

A. En el bloque `1. VARIABLES GLOBALES` (sección `//--- pista de carreras y naves ---`)

C++

```cpp
struct NodoPista {
        glm::vec3 pos;    // Posición en coordenadas de pista.obj
        glm::vec3 normal; // Vector normal del piso en ese punto
};

// Pega aquí los puntos generados por el script de Blender
// (Si el orden sale en sentido contrario al avance, invierte el vector con std::rev
std::vector<NodoPista> rutaPista = {
        // Ejemplo de estructura (reemplazar con el output de Blender):
        { glm::vec3(0.0f, 0.0f,   0.0f), glm::vec3(0.0f, 1.0f, 0.0f) },
        { glm::vec3(0.0f, 0.0f, -50.0f), glm::vec3(0.0f, 1.0f, 0.0f) },
        // ...
};

std::vector<float> distAcumuladaPista;
float longitudTotalPista = 0.0f;
float avanceCarrera = 0.0f;
float velocidadCarrera = 2.5f; // Ajusta la velocidad de las naves aquí
```

C++

```cpp
// Se llama una sola vez en main() antes del while
void InicializarRutaPista()
{
        int n = rutaPista.size();
        if (n < 2) return;

        distAcumuladaPista.resize(n + 1, 0.0f);
        longitudTotalPista = 0.0f;

        for (int i = 0; i < n; i++)
        {
                glm::vec3 pActual = rutaPista[i].pos * ESCALA_INTERNA_PISTA;
                glm::vec3 pSig = rutaPista[(i + 1) % n].pos * ESCALA_INTERNA_PISTA;
                longitudTotalPista += glm::distance(pActual, pSig);
                distAcumuladaPista[i + 1] = longitudTotalPista;
        }
}

// Devuelve la matriz local respecto a modelaux para cualquier nave según su distanc
glm::mat4 ObtenerMatrizNavePista(float distancia, float carrilX, float alturaFlotacio
{
        int n = rutaPista.size();
        if (n < 2 || longitudTotalPista <= 0.0f) return glm::mat4(1.0f);

        // Mantener la distancia siempre dentro del circuito cerrado [0, longitudTot
        float d = fmod(distancia, longitudTotalPista);
        if (d < 0.0f) d += longitudTotalPista;

        // Buscar en qué segmento se encuentra la nave
        int idx = 0;
        for (int i = 0; i < n; i++)
        {
                if (d >= distAcumuladaPista[i] && d <= distAcumuladaPista[i + 1])
                {
                        idx = i;
                        break;
                }
        }

        int idxSig = (idx + 1) % n;
        int idxSig2 = (idx + 2) % n;

        float longSeg = distAcumuladaPista[idx + 1] - distAcumuladaPista[idx];
        float t = (longSeg > 0.0001f) ? (d - distAcumuladaPista[idx]) / longSeg : 0.

        // 1. Interpolar posición central y vector normal del suelo
        glm::vec3 p0 = rutaPista[idx].pos * ESCALA_INTERNA_PISTA;
        glm::vec3 p1 = rutaPista[idxSig].pos * ESCALA_INTERNA_PISTA;
        glm::vec3 p2 = rutaPista[idxSig2].pos * ESCALA_INTERNA_PISTA;

        glm::vec3 posCentro = glm::mix(p0, p1, t);
        glm::vec3 normalPiso = glm::normalize(glm::mix(rutaPista[idx].normal, rutaPi
```

```cpp
        // 2. Interpolar la tangente de avance (Frente) para que el giro entre tramo
        glm::vec3 tangenteActual = glm::normalize(p1 - p0);
        glm::vec3 tangenteSig = glm::normalize(p2 - p1);
        glm::vec3 frente = glm::normalize(glm::mix(tangenteActual, tangenteSig, t));

        // 3. Construir el marco ortonormal (Derecha, Arriba/Normal, Frente)
        glm::vec3 derecha = glm::normalize(glm::cross(frente, normalPiso));
        glm::vec3 arriba = glm::normalize(glm::cross(derecha, frente));

        // 4. Calcular posición final desplazando hacia el carril lateral y elevando
        glm::vec3 posFinal = posCentro + (derecha * carrilX) + (arriba * alturaFlota

        // 5. Armar matriz 4x4 (Como las naves miran hacia -Z, el eje +Z local es -f
        glm::mat4 transform(1.0f);
        transform[0] = glm::vec4(derecha, 0.0f); // Columna 0: Eje X local (Peralte
        transform[1] = glm::vec4(arriba,  0.0f); // Columna 1: Eje Y local (Normal d
        transform[2] = glm::vec4(-frente, 0.0f); // Columna 2: Eje Z local (Direcció
        transform[3] = glm::vec4(posFinal, 1.0f);// Columna 3: Traslación

        return transform;
}
```

```cpp
// Avanzar toda la formación de naves cada frame
avanceCarrera += velocidadCarrera * deltaTime;
if (longitudTotalPista > 0.0f && avanceCarrera > longitudTotalPista)
        avanceCarrera -= longitudTotalPista;

// Arreglo con punteros a tus 8 modelos en el orden exacto de la parrilla de salida
Model* navesFZero[8] = {
        &blueFalcon,     &fireStingray,
        &goldenFox,      &wildGoose,
        &nightThunder,   &rainbowPhoenix,
        &redGazzelle,    &blackBull
};

for (int i = 0; i < 8; i++)
{
        // Los pares (0, 2, 4, 6) van en el carril izquierdo (-7.15f)
        // Los impares (1, 3, 5, 7) van en el carril derecho (+7.15f)
        float carrilX = (i % 2 == 0) ? -7.15f : 7.15f;

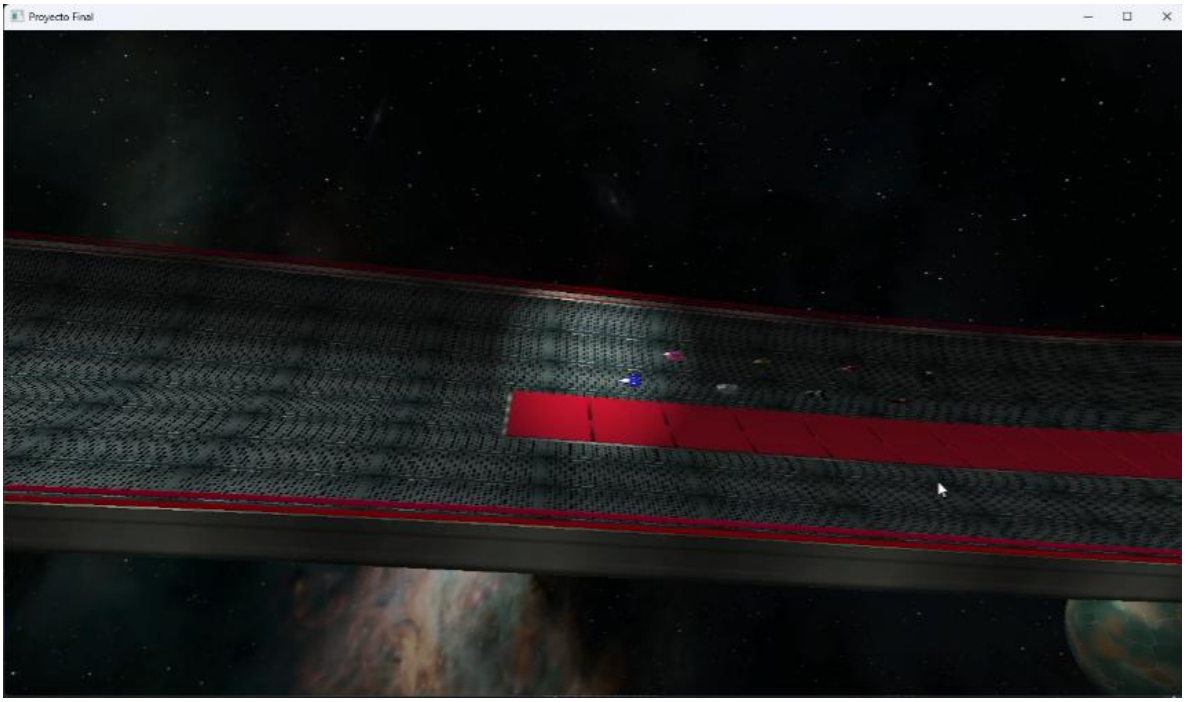
        // Desfase longitudinal: -10, -20, -30... hasta -80 detrás del líder
        float desfaseAtras = -10.0f * (i + 1);

        // Pequeña oscilación senoidal para dar el efecto de levitación antigravedad
        float flotacion = 1.00f + 0.15f * sin((avanceCarrera * 0.5f) + i);

        model = modelaux * ObtenerMatrizNavePista(avanceCarrera + desfaseAtras, carr

        color = glm::vec3(1.0f, 1.0f, 1.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        navesFZero[i]->RenderModel();
}
```

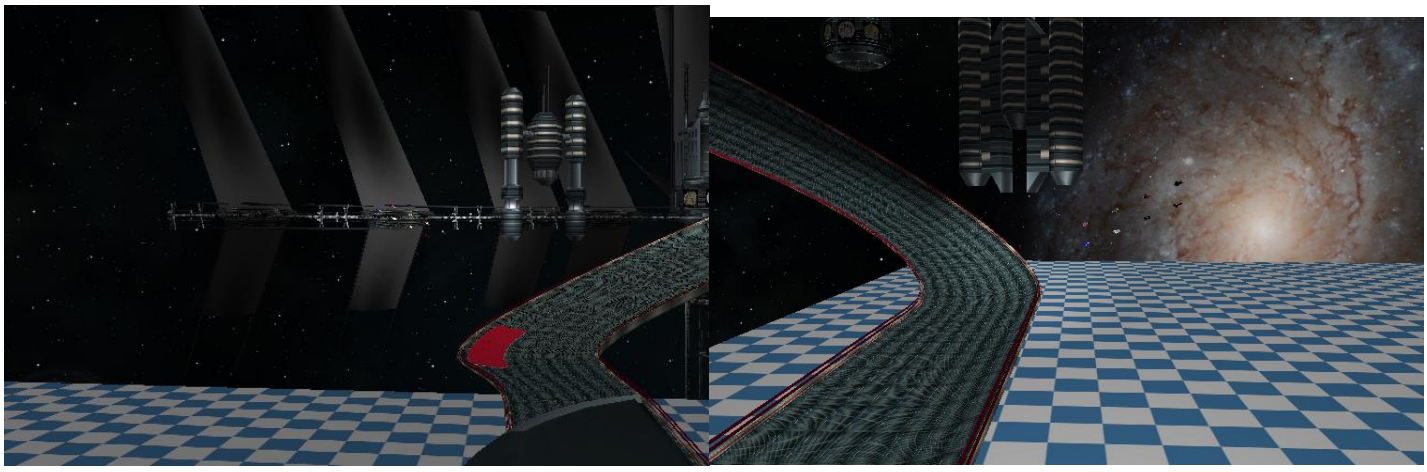
Una vez implementados las estructuras de control y cambiado el ciclo de dibujado se obtuvieron los carros moviéndose en pista siguiendo los waypoints capturados en blender.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Humo MUCHOS PERO MUCHOS problemas a la hora de implementar los waypoints. OpenGL hace la lectura del array proporcionado al revés entonces los carros avanzaban hacia atrás al arrancar desde la meta. Para ello se reordenaron los datos del array pasando el primero al último el penúltimo al segundo y así sucesivamente.

También había un offset que se tuvo que calcular poco a poco al tanteo para dar un buen efecto. Como ventaja se tiene que las maquinas no van directamente haciendo contacto con el suelo si no que levitan encima del mismo aproximadamente una unidad lo cual ayuda a mantener la ilusión del movimiento y maquilla un poco el clipping de los modelos en el entorno.



3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Esta practica me pareció muy adecuada en su aplicación ya que se hace uso de conceptos que genuinamente no pensé que volvería a emplear o que ya los tenía olvidados. El tiro parabólico y las ecuaciones de la recta, circunferencia, elipse, etc son fundamentos teóricos que trabajé hace tiempo en la DCB y m entusiasmo usarlo en una rama de la computación.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Creo que solo falto hacer comentarios sobre los métodos alternativos de animación. Es verdad que existen tutoriales de blender y openGL sobre técnicas de animación que van más alla de la definición de básica y compleja vista en clase y visitarlos puede ser enriquecedor para complementar la practica.

c. Conclusión

Está practica me ayudo a entender los principios que permiten realizar animación de objetos en openGL. Haciendo uso de rotaciones y traslaciones se puede controlar el movimiento de un objeto en el espacio sin necesidad de que el usuario presione una tecla directamente para hacer el movimiento como se había visto anteriormente. También es importante el concepto del delta time revisado durante la exposición practica y ejercicios ya que sin el las animaciones no sería consistentes en diferentes equipos con distinto hardware.